07 DEPENDENCY INJECTION (DI) IN NESTJS

Dependency Injection is a design pattern where a class does NOT create the objects it depends on itself — instead, something else (a "container") creates them and hands them over. In NestJS, that container is built into the framework and runs automatically.

```
WITHOUT DI (you manage everything yourself):
  class UserController {
    private userService = new UserService();   // manual creation
  }

WITH DI (Nest manages it for you):
  class UserController {
    constructor(private readonly userService: UserService) {}
    // Nest sees this parameter and creates/injects UserService itself
  }
```

1. THE PROBLEM DI SOLVES

If a class creates its own dependencies with "new", you get:

- Tight coupling — UserController is now permanently tied to exactly one UserService implementation.
- Hard to test — you can't swap in a fake/mock UserService for a unit test without changing UserController's code.
- Hard to share — if ten different classes all need a UserService, you either create ten separate instances (wasteful) or build your own manual singleton/registry system (reinventing what Nest already does).

DI flips this around: a class simply DECLARES what it needs (via the constructor), and something external is responsible for supplying it.

2. HOW IT WORKS IN THIS PROJECT (REAL EXAMPLE)

```ts
// user.service.ts
import { Injectable } from '@nestjs/common';

@Injectable()
export class UserService {
  getHello(): string {
    return 'i am sinchana';
  }
}

// user.controller.ts
import { Controller, Get } from '@nestjs/common';
import { UserService } from './user.service';

@Controller('user')
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Get()
```

```
  getHello(): string {
    return this.userService.getHello();
  }
}

// app.module.ts
@Module({
  controllers: [AppController, UserController],
  providers: [AppService, UserService],   // <-- registered here
})
export class AppModule {}
```

Step by step, what actually happens at startup:

1. @Injectable() marks UserService as something Nest's DI container is ALLOWED to manage.
2. Listing UserService in the module's "providers" array tells Nest "this dependency exists and you're responsible for it."
3. Nest sees UserController's constructor asks for a UserService (by its TypeScript type).
4. Nest creates ONE instance of UserService, and passes it into UserController's constructor automatically.
5. You never write "new UserService()" anywhere — Nest owns that job.

If step 2 were skipped (UserService not in "providers"), Nest would throw a runtime error at startup like:

> "Nest can't resolve dependencies of the UserController (?). Please make sure that the argument UserService is available..." This is exactly the bug from earlier in this project — UserController imported UserService and asked for it in the constructor, but it hadn't been registered as a provider yet.

3. WHY IT'S CALLED "INJECTION"

The dependency (UserService) is "injected" INTO UserController from the outside, rather than UserController reaching out and constructing it itself. Nest is the "injector."

This specific style — declaring a dependency as a constructor parameter and letting the framework supply it — is called CONSTRUCTOR INJECTION, and it's the primary (and recommended) form of DI in Nest.

4. THE THREE PIECES THAT MAKE DI WORK

```
@Injectable()
  Marks a class as a "provider" — eligible to be managed and handed
  out by Nest's DI container.
```

> providers: [...] (in a @Module) Registers a provider with a specific module's DI container so Nest knows it exists and can resolve requests for it.
>
> constructor(private readonly x: SomeService) Declares a dependency. Nest inspects the constructor's parameter TYPES (via TypeScript's reflection metadata) to figure out exactly which provider to inject.

5. SINGLETON BY DEFAULT

By default, Nest creates exactly ONE instance of each provider and reuses it everywhere it's injected across the whole app (this is called "singleton scope"). So if both UserController and, say, OrdersController inject

UserService, they share the SAME instance — not two separate copies.

This matters for state: if a service holds in-memory data (like an array), every injector sees the same shared data.

6. WHY THIS MATTERS — THE TESTING PAYOFF

Because UserController only asks for "something that satisfies the UserService type" rather than creating it directly, tests can swap in a fake version:

```
const module: TestingModule = await Test.createTestingModule({
  controllers: [UserController],
  providers: [
    {
      provide: UserService,
      useValue: { getHello: () => 'fake response' },  // mock!
    },
  ],
}).compile();
```

UserController's code doesn't change at all — Nest just hands it the fake object instead of a real UserService. This is the main practical reason DI exists: it decouples "what a class needs" from "how that need gets fulfilled."

7. DI BEYOND SERVICES (IT'S NOT JUST @Injectable CLASSES)

Nest's DI container can also inject:

- Configuration values (via ConfigService)
- Database repositories (e.g. @InjectRepository(User) in TypeORM)
- Values by a custom token instead of a class type:

```
// registering a custom token
providers: [{ provide: 'APP_NAME', useValue: 'my-app' }]

// injecting it
constructor(@Inject('APP_NAME') private appName: string) {}
```

@Inject('TOKEN') is needed here because a plain string/interface has no runtime type Nest can auto-detect — @Inject explicitly tells Nest which provider to match.

★ RULE OF THUMB

If a class needs another class to do its job, don't write "new OtherClass()" inside it. Add it as a constructor parameter, mark the other class @Injectable(), and list it in the module's providers array. Let Nest wire it together — that's the entire point of DI.

See also: theory/02-controller.txt, theory/03-service.txt, theory/06-decorators.txt